RK3506 System Suspend Developer Guide

ID: RK-KF-YF-C26

Release Version: V1.3.0

Release Date: 2026-05-29

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is used to guide users on how to configure the RK3506 system suspend mode according to product requirements.

Product Version

Chipset	Kernel Version
RK3506	6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Jianhong Chen	2025-01-14	Initial Version
V1.1.0	Jianhong Chen	2025-03-06	Content Improvement
V1.2.0	Jianhong Chen	2026-03-24	Add troubleshooting section
V1.3.0	Jianhong Chen	2026-05-29	Add GPIO troubleshooting section

Contents

RK3506 System Suspend Developer Guide

- 1. System suspend
  - 1.1 Driver Files
  - 1.2 DTS Nodes
- 2. DTS Configuration
  - 2.1 Basic Configuration
  - 2.2 Power Configuration
  - 2.3 Wake-up Source Configuration
  - 2.4 GPIO Configuration
  - 2.5 Debug Configuration
  - 2.6 LOGIC Power-off Configuration
- 3. Print Information
- 4. GPIO Troubleshooting
- 5. Troubleshooting

1. System suspend

The suspend process on the Linux platform: The general suspend process of the Linux kernel (completing peripheral modules, OS Suspend) ==> Entering the OPTEE of RK3506 to complete the suspend process of the SoC.

Firmware Version Description: The OPTEE firmware corresponds to the rk3506_tee_vx.yy.bin file in the rkbin repository. Considering aspects such as the perfection of functions and the repair of BUGs, it is recommended that the version used by customers is not lower than:

```
rk3506_tee_v2.10.bin      // Default for sdk
rk3506_tee_ta_v1.10.bin   // Used when customers have ta requirements
```

The suspend process in OPTEE generally includes the following operations: shutting down module IPs, clocks, PLLs, entering self-refresh for DDR, switching the system bus to a low-speed clock (24M or 32K), powering off vdd_arm/vdd_logic, configuring wake-up sources, etc. To meet the suspend mode requirements of different products, the relevant configurations are currently passed to OPTEE through DTS nodes during the boot phase.

Note: There is no correlation or reference between the suspend configuration options/methods of different platforms; this document is only applicable to the RK3506 platform. For the concepts of "VD_PMU, VD_Logic domain" mentioned in the current document, please refer to the PMU section of the TRM.

1.1 Driver Files

```
./drivers/soc/Rockchip/rockchip_pm_config.c
./drivers/firmware/Rockchip_sip.c
./include/dt-bindings/suspend/rockchip-rk3506.h
```

1.2 DTS Nodes

```
rockchip_suspend: rockchip-suspend {
    compatible = "rockchip,pm-rk3506";
    status = "okay";
    // Basic configuration (necessary, configure according to actual requirements)
    rockchip,sleep-mode-config = <
        (0
        | RKPM_ARMOFF_DDRPD
        | RKPM_24M_OSC_DIS
        | RKPM_32K_CLK
        | RKPM_32K_SRC_RC
        )
    >;
```

```
    // Wake-up source configuration (necessary, configure according to actual requirements)
    rockchip,wakeup-config = <
        (0
        | RKPM_GPIO0_WAKEUP_EN
        )
    >;
    // Power configuration (necessary, configure according to actual hardware design)
    rockchip,apios-suspend = <
        (0
        | RKPM_PWREN_CORE_GPIO0A2 | RKPM_PWREN_CORE_ACT_LOW
        | RKPM_PWREN_SLEEP_GPIO0A0 | RKPM_PWREN_SLEEP_ACT_LOW
        )
    >;
    // GPIO configuration (optional, configure according to actual requirements. Used to
configure peripheral IO status to reduce suspend power consumption)
#define GPIO0_IOC_GPIO0A_PULL_REG   0xff950200
#define GPIO0_IOC_GPIO0C_PULL_REG   0xff950208
#define GPIO0A3_PULL_DOWN       0x00c00080
#define GPIO0C5_PULL_DOWN       0x0c000800
    rockchip,sleep-io-config = <
        GPIO0_IOC_GPIO0C_PULL_REG GPIO0C5_PULL_DOWN /* PWM0_CH0_CPU */
        GPIO0_IOC_GPIO0A_PULL_REG GPIO0A3_PULL_DOWN /* PWM0_CH2_LCD_BL */
    >;
    // Debug configuration (optional, configure according to actual requirements. Used for
debugging and analysis)
    rockchip,sleep-debug-en = <
        (0
        | RKPM_DBG_TIMEOUT_S(3)
        )
    >;
};
```

The configuration provided in the Rockchip SDK is generally universal and comprehensive, serving as a reference. Customers should adjust and modify it according to their specific needs.

2. DTS Configuration

Currently supported configuration options are defined in:

```
./include/dt-bindings/suspend/rockchip-rk3506.h
```

2.1 Basic Configuration

Configuration items:

```
rockchip,sleep-mode-config = <...>;
```

Configuration source:

```
// ARM power supply is not cut off. Generally not used in products, only for
troubleshooting.
#define RKPM_ARMPD                      BIT(0)

// ARM power supply cut off + ddr does not enter low power consumption. Generally not used
in products, only for troubleshooting.
#define RKPM_ARMOFF                     BIT(1)

// ARM power supply cut off + ddr enters low power consumption.
#define RKPM_ARMOFF_DDRPD               BIT(2)

// ARM power supply cut off + Logic power supply cut off + ddr enters low power
consumption.
#define RKPM_ARMOFF_LOGOFF              BIT(3)

// Disable the 24M crystal oscillator (only allowed when switching to 32k clock).
#define RKPM_24M_OSC_DIS                BIT(4)

// System clock switches to 32K.
#define RKPM_32K_CLK                    BIT(5)
// The source of the 32K system clock (RKPM_32K_CLK must be configured). They are:
// - External 32k crystal oscillator (board support required)
// - SoC internal RC400 clock divided down to 32k
// - 24M crystal oscillator divided (RKPM_24M_OSC_DIS cannot be configured at this time)
// If accuracy in suspend power consumption is a concern, the recommendation is: External
32k crystal oscillator > 24M crystal oscillator division (suspend cannot turn off the
crystal oscillator, power consumption will be higher than choosing RC clock) > SoC internal
RC clock.
// If accuracy in suspend power consumption is not a concern, the recommendation is: SoC
internal RC clock.
#define RKPM_32K_SRC_OSC                BIT(6)
#define RKPM_32K_SRC_RC                 BIT(7)
#define RKPM_32K_SRC_24XIN              BIT(8)

// Use separate power supply scheme and VDD_ARM uses PWM0_CH0 to control PWM voltage
regulation, it needs to be configured.
#define RKPM_PWM0_CH0_REGULATOR         BIT(9)

// Do not cut off PWM0 clock, corresponding to cru gate1[1]. If the breathing light on pwm0
needs to continue working during suspend, it needs to be configured.
#define RKPM_PWM0_SRC_24M_EN            BIT(10)

// Do not cut off PWM0 clock, corresponding to cru gate1[0]. Generally not configured in
products, only used for special requirements or troubleshooting.
#define RKPM_PWM0_SRC_PLL_EN            BIT(11)

// Do not cut off the total clock of the VD_Logic domain. Generally not configured in
products, only used for special requirements or troubleshooting.
#define RKPM_LOG_CLK_ALIVE              BIT(12)
```

```
// Do not cut off PLL. Generally not configured in products, only used for special
requirements or troubleshooting.
#define RKPM_GPLL_ALIVE                 BIT(13)
#define RKPM_V0PLL_ALIVE                BIT(14)
#define RKPM_V1PLL_ALIVE                BIT(15)

// Disable GPIO4 input enable. There may be some benefits in suspend power consumption, but
GPIO4 cannot be used as a wake-up source.
// Customers need to verify and confirm whether to use it.
#define RKPM_GPIO4_IE_DIS               BIT(16)

// VD_PMU domain modules do not switch to low-speed clock. Generally needed when VD_PMU
domain modules need to work normally during suspend.
// For example: When the system is in suspend, the MCU also continues to work, and it needs
to control the pin state on GPIO0.
#define RKPM_PMU_BUS_CLK_HS             BIT(17)

// Only configured on the RK801 power supply scheme.
#define RKPM_PWM0_CH0_CORE_PWREN        BIT(18)
```

Currently, the RK3506 supports two suspend modes:

- ARM power supply cut off.
- ARM+LOGIC power supply cut off.

2.2 Power Configuration

Configuration Items:

```
rockchip,apios-suspend = <...>;
```

Configuration Source:

```
// PWRCTRL_SLEEP pin polarity when power is turned off
#define RKPM_PWREN_SLEEP_ACT_LOW        PIN0(PWREN_ACT_LOW_MASK)
#define RKPM_PWREN_SLEEP_ACT_HIGH       PIN0(PWREN_ACT_HIGH_MASK)
// PWRCTRL_SLEEP pin definition
#define RKPM_PWREN_SLEEP_GPIO0A0        PIN0(0)
#define RKPM_PWREN_SLEEP_GPIO0A1        PIN0(1)
#define RKPM_PWREN_SLEEP_GPIO0A2        PIN0(2)
#define RKPM_PWREN_SLEEP_GPIO0A3        PIN0(3)
#define RKPM_PWREN_SLEEP_GPIO0A4        PIN0(4)
#define RKPM_PWREN_SLEEP_GPIO0A5        PIN0(5)
#define RKPM_PWREN_SLEEP_GPIO0A6        PIN0(6)
#define RKPM_PWREN_SLEEP_GPIO0A7        PIN0(7)
#define RKPM_PWREN_SLEEP_GPIO0B0        PIN0(8)
#define RKPM_PWREN_SLEEP_GPIO0B1        PIN0(9)
#define RKPM_PWREN_SLEEP_GPIO0B2        PIN0(10)
#define RKPM_PWREN_SLEEP_GPIO0B3        PIN0(11)
#define RKPM_PWREN_SLEEP_GPIO0B4        PIN0(12)
```

```
#define RKPM_PWREN_SLEEP_GPIO0B5         PIN0(13)
#define RKPM_PWREN_SLEEP_GPIO0B6         PIN0(14)
#define RKPM_PWREN_SLEEP_GPIO0B7         PIN0(15)
#define RKPM_PWREN_SLEEP_GPIO0C0         PIN0(16)
#define RKPM_PWREN_SLEEP_GPIO0C1         PIN0(17)
#define RKPM_PWREN_SLEEP_GPIO0C2         PIN0(18)
#define RKPM_PWREN_SLEEP_GPIO0C3         PIN0(19)
#define RKPM_PWREN_SLEEP_GPIO0C4         PIN0(20)
#define RKPM_PWREN_SLEEP_GPIO0C5         PIN0(21)
#define RKPM_PWREN_SLEEP_GPIO0C6         PIN0(22)
#define RKPM_PWREN_SLEEP_GPIO0C7         PIN0(23)
#define RKPM_PWREN_SLEEP_GPIO1B1         PIN0(24)
#define RKPM_PWREN_SLEEP_GPIO1B2         PIN0(25)
#define RKPM_PWREN_SLEEP_GPIO1B3         PIN0(26)
#define RKPM_PWREN_SLEEP_GPIO1C2         PIN0(27)
#define RKPM_PWREN_SLEEP_GPIO1C3         PIN0(28)
#define RKPM_PWREN_SLEEP_GPIO1D1         PIN0(29)
#define RKPM_PWREN_SLEEP_GPIO1D2         PIN0(30)
#define RKPM_PWREN_SLEEP_GPIO1D3         PIN0(31)

// PWRCTRL_CORE pin polarity when power is turned off
#define RKPM_PWREN_CORE_ACT_LOW          PIN8(PWREN_ACT_LOW_MASK)
#define RKPM_PWREN_CORE_ACT_HIGH         PIN8(PWREN_ACT_HIGH_MASK)
// PWRCTRL_CORE pin definition
#define RKPM_PWREN_CORE_GPIO0A0          PIN8(0)
#define RKPM_PWREN_CORE_GPIO0A1          PIN8(1)
#define RKPM_PWREN_CORE_GPIO0A2          PIN8(2)
#define RKPM_PWREN_CORE_GPIO0A3          PIN8(3)
#define RKPM_PWREN_CORE_GPIO0A4          PIN8(4)
#define RKPM_PWREN_CORE_GPIO0A5          PIN8(5)
#define RKPM_PWREN_CORE_GPIO0A6          PIN8(6)
#define RKPM_PWREN_CORE_GPIO0A7          PIN8(7)
#define RKPM_PWREN_CORE_GPIO0B0          PIN8(8)
#define RKPM_PWREN_CORE_GPIO0B1          PIN8(9)
#define RKPM_PWREN_CORE_GPIO0B2          PIN8(10)
#define RKPM_PWREN_CORE_GPIO0B3          PIN8(11)
#define RKPM_PWREN_CORE_GPIO0B4          PIN8(12)
#define RKPM_PWREN_CORE_GPIO0B5          PIN8(13)
#define RKPM_PWREN_CORE_GPIO0B6          PIN8(14)
#define RKPM_PWREN_CORE_GPIO0B7          PIN8(15)
#define RKPM_PWREN_CORE_GPIO0C0          PIN8(16)
#define RKPM_PWREN_CORE_GPIO0C1          PIN8(17)
#define RKPM_PWREN_CORE_GPIO0C2          PIN8(18)
#define RKPM_PWREN_CORE_GPIO0C3          PIN8(19)
#define RKPM_PWREN_CORE_GPIO0C4          PIN8(20)
#define RKPM_PWREN_CORE_GPIO0C5          PIN8(21)
#define RKPM_PWREN_CORE_GPIO0C6          PIN8(22)
#define RKPM_PWREN_CORE_GPIO0C7          PIN8(23)
#define RKPM_PWREN_CORE_GPIO1B1          PIN8(24)
#define RKPM_PWREN_CORE_GPIO1B2          PIN8(25)
#define RKPM_PWREN_CORE_GPIO1B3          PIN8(26)
#define RKPM_PWREN_CORE_GPIO1C2          PIN8(27)
#define RKPM_PWREN_CORE_GPIO1C3          PIN8(28)
```

```
#define RKPM_PWREN_CORE_GPIO1D1          PIN8(29)
#define RKPM_PWREN_CORE_GPIO1D2          PIN8(30)
#define RKPM_PWREN_CORE_GPIO1D3          PIN8(31)
```

The RK3506 hardware design currently supports the use of one or two pwrctrl pins to control the suspend state of vdd_core, vdd_logic, and other power supplies. Please configure according to the actual hardware design and suspend requirements of the product.

For example, the configuration in `arch/arm/boot/dts/rk3502-evb1-v10.dtsi`:

```
rockchip,sleep-mode-config = <
                (0
                | RKPM_ARMOFF_DDRPD
                | RKPM_24M_OSC_DIS
                | RKPM_32K_CLK
                | RKPM_32K_SRC_RC
                | RKPM_PWM0_CH0_REGULATOR
                )
        >;

// - pwrctrl_core(gpio0_a2), power off vdd_core when pulled low.
// - pwrctrl_sleep(gpio0_a0), power off vdd_logic and other power supplies when pulled low.
// - Note: Although both pwrctrl pins are configured here, only pwrctrl_core is pulled in
RKPM_ARMOFF_DDRPD suspend mode, and both pwrctrl pins are pulled only in RKPM_ARMOFF_LOGOFF
mode.
rockchip,apios-suspend = <
        (0
        | RKPM_PWREN_CORE_GPIO0A2 | RKPM_PWREN_CORE_ACT_LOW
        | RKPM_PWREN_SLEEP_GPIO0A0 | RKPM_PWREN_SLEEP_ACT_LOW
        )
>;
```

2.3 Wake-up Source Configuration

Configuration items:

```
rockchip,wakeup-config = <...>;
```

Configuration sources:

```
// Not supported yet. Please raise a Redmine ticket if needed.
#define RKPM_ARM_WAKEUP_EN               BIT(0)
#define RKPM_M0_WAKEUP_EN                BIT(1)
#define RKPM_HPTIMER_WAKEUP_EN           BIT(2)

// Supported.
#define RKPM_GPIO0_WAKEUP_EN             BIT(3)
#define RKPM_GPIO1_SHADOW_WAKEUP_EN      BIT(4)
#define RKPM_GPIO2_WAKEUP_EN             BIT(5)
#define RKPM_GPIO3_WAKEUP_EN             BIT(6)
```

```
#define RKPM_GPIO4_WAKEUP_EN            BIT(7)

// Not supported yet. Please raise a Redmine ticket if needed.
#define RKPM_PWM0_WAKEUP_EN             BIT(8)
#define RKPM_PWM1_WAKEUP_EN             BIT(9)
#define RKPM_PWM2_WAKEUP_EN             BIT(10)

// Supported. Configuration notes for UART as a wake-up source:
// - suspend mode selection: vdd_logic should not be powered off, and the clock should not be
switched to 32k.
// - Kernel uart node: Add "wakeup-source;" attribute.
// - The uart clock source should be a 24M crystal, ensuring that the uart clock is not
turned off before suspend.
#define RKPM_UART0_WAKEUP_EN            BIT(11)
#define RKPM_UART1_WAKEUP_EN            BIT(12)
#define RKPM_UART2_WAKEUP_EN            BIT(13)
#define RKPM_UART3_WAKEUP_EN            BIT(14)
#define RKPM_UART4_WAKEUP_EN            BIT(15)
#define RKPM_UART5_WAKEUP_EN            BIT(16)

// Supported. (The system clock can be switched to 32K mode during suspend)
#define RKPM_USB0_DET_WAKEUP_EN         BIT(17)
#define RKPM_USB1_DET_WAKEUP_EN         BIT(18)

// Not supported yet. Please raise a Redmine ticket if needed.
#define RKPM_MAC0_WAKEUP_EN             BIT(19)
#define RKPM_MAC1_WAKEUP_EN             BIT(20)
#define RKPM_TIMER0_WAKEUP_EN           BIT(21)
#define RKPM_TIMER1_WAKEUP_EN           BIT(22)
#define RKPM_CAN0_WAKEUP_EN             BIT(23)
#define RKPM_CAN1_WAKEUP_EN             BIT(24)
#define RKPM_TOUCH_KEY_WAKEUP_EN        BIT(25)
#define RKPM_SARADC_WAKUP_EN            BIT(26)
#define RKPM_ALARM_WAKEUP_EN            BIT(27)
#define RKPM_SECOND_CATCH_WAKEUP_EN     BIT(28)

// Supported.
#define RKPM_PMU_TIMEOUT_EN             BIT(29)
```

Wake-up source notes:

- In the case of ARM+LOGIC power-off:

  Wake-up sources only support VD_PMU domain module interrupts, mainly GPIO0, GPIO1_Shadow.
- For USB wake-up, please confirm that the kernel repository includes the following patch:

```
commit 8e6926b1d4ae8ace396f5bd59e3e7801ca2ee98f
Author: Jianwei Zheng <jianwei.zheng@rock-chips.com>
Date:   Fri Nov 15 16:37:45 2024 +0800

    phy: rockchip: inno-usb2: support usb wakeup for rk3506

    1. Support linestate filter time control register for OTG0 and OTG1.
    2. Disable disconnect rise and disconnect fall irq. Because these
    two interrupts will be triggered immediately after the system goes
    into sleep mode, causing the system to be unable to sleep.

    Change-Id: Ia6c1e58ef926f86dee6df0013aafe9bfe4586b52
    Signed-off-by: Jianwei Zheng <jianwei.zheng@rock-chips.com>
```

2.4 GPIO Configuration

Configuration items:

```
rockchip,sleep-io-config = <...>;
```

This configuration item is primarily used to optimize suspend power consumption.

The suspend power consumption requirements for current products are relatively high, and unreasonable GPIO states (input, output, pull-up/pull-down, etc.) can increase suspend power consumption to some extent. For these types of GPIOs, if it is not convenient to set the GPIO state during suspend in the corresponding driver code, then configuration can be done here. The configuration specified here will be set at the final stage of the suspend process, and the registers will be restored to their pre-sleep state upon wake-up. Please configure as needed in conjunction with your own hardware design.

Configuration is done using key-value pairs: "Register address Configuration value", with a maximum support of 16 sets. However, only the following can be configured:

- GPIO controller's GPIO_SWPORT_DR and GPIO_SWPORT_DDR registers.
- All registers of the IOC controller.

If the key-value pair is invalid, the suspend print will indicate "(invalid)" and ignore the configuration;

If the key-value pair exceeds 16 sets, the suspend print will indicate "(overflow)" and ignore the configuration.

2.5 Debug Configuration

Configuration items:

```
rockchip,sleep-debug-en = <...>;
```

Not used in products, only for troubleshooting.

Configuration source:

```
// Enable all clocks when in suspend.
#define RKPM_DBG_CLK_UNGATE              BIT(0)

// Keep all PLLs always on when in suspend.
#define RKPM_DBG_PLLS_ALIVE              BIT(1)

// Ignore.
#define RKPM_DBG_PMU_SOUT                BIT(2)

// Print register values before executing the SoC suspend process.
#define RKPM_DBG_REG1                    BIT(3)

// Print register values after executing the SoC suspend process.
#define RKPM_DBG_REG2                    BIT(4)

// Ignore.
#define RKPM_DBG_DEEP_JTAG               BIT(5)
#define RKPM_DBG_JTAG                    BIT(6)

// PMU internal timer automatically wakes up after a timeout (unit: seconds), generally used
only for testing sleep and wake-up during the development phase.
#define RKPM_DBG_TIMEOUT_S(i)            (((i) & 0xf) << 7)

// Let the CPU enter low power before executing the SoC suspend process.
// Used to test scenarios where interrupts cannot wake up the system, with the purpose of
confirming whether the failure of interrupts is due to the configuration of the SoC suspend
process.
#define RKPM_DBG_DUMMY_WFI               BIT(11)

// Insert two 5s delays at the beginning and end of the SoC suspend process. Used to confirm
whether certain exceptions are caused by the SoC suspend process.
#define RKPM_DBG_DUMMY_DELAY1            BIT(12)
#define RKPM_DBG_DUMMY_DELAY2            BIT(13)

// Similar to RKPM_DBG_REG1/2, print out more peripheral module register values.
#define RKPM_DBG_PERI_REG                BIT(14)
```

- RKPM_DBG_REG1: Equivalent to printing the register status after the kernel completes all suspend processes.
- RKPM_DBG_REG2: Equivalent to printing the register status before OP-TEE completes the final sleep step. If you want to analyze the I/O power consumption during suspend, this configuration item should be used as the reference.

2.6 LOGIC Power-off Configuration

To accommodate the suspend scenarios of most product forms, the SDKs released by default are configured for ARM power-off mode.

Below are the corresponding ARM+LOGIC power-off mode configurations for the reference boards.

- RK3506G-EVB1 board:

```
diff --git a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi b/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
index 852840cc78c..69124f89661 100644
--- a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
@@ -396,7 +396,7 @@
 &rockchip_suspend {
        rockchip,sleep-mode-config = <
                (0
-               | RKPM_ARMOFF_DDRPD
+               | RKPM_ARMOFF_LOGOFF
                | RKPM_24M_OSC_DIS
                | RKPM_32K_CLK
                | RKPM_32K_SRC_RC
```

- RK3506G-DEMO(RK801) board:

```
diff --git a/arch/arm/boot/dts/rk3506-rk801.dtsi b/arch/arm/boot/dts/rk3506-rk801.dtsi
index a0c0778a214..5028e00031b 100644
--- a/arch/arm/boot/dts/rk3506-rk801.dtsi
+++ b/arch/arm/boot/dts/rk3506-rk801.dtsi
@@ -104,7 +104,7 @@
                                regulator-boot-on;
                                regulator-always-on;
                                regulator-state-mem {
-                                       regulator-on-in-suspend;
+                                       regulator-off-in-suspend;
                                        regulator-suspend-microvolt = <1800000>;
                                };
                        };
@@ -114,7 +114,7 @@
                                regulator-boot-on;
                                regulator-always-on;
                                regulator-state-mem {
-                                       regulator-on-in-suspend;
+                                       regulator-off-in-suspend;
                                };
                        };
                };
diff --git a/arch/arm/boot/dts/rk3506g-demo-display-control.dts b/arch/arm/boot/dts/rk3506g-demo-display-control.dts
index 59f8ade8a32..f982c38bd89 100644
--- a/arch/arm/boot/dts/rk3506g-demo-display-control.dts
+++ b/arch/arm/boot/dts/rk3506g-demo-display-control.dts
@@ -580,7 +580,7 @@
 &rockchip_suspend {
        rockchip,sleep-mode-config = <
                (0
-               | RKPM_ARMOFF_DDRPD
+               | RKPM_ARMOFF_LOGOFF
```

```
                | RKPM_24M_OSC_DIS
                | RKPM_32K_CLK
                | RKPM_32K_SRC_RC
```

3. Print Information

Here is a brief introduction to the meaning of print information when the system is in suspend and when it wakes up. For the convenience of annotation, some print content is broken into lines, and different suspend power consumption modes will also bring different prints. All print information content is subject to the actual display.

RK3506 System suspend Print:

```
// Version information. Where: "fwver: v1.10" is the firmware version number; "-gxxxxx" is
the commit number of the firmware compilation point.
3.13.0-885-gf24e46b4e16 #chenjh (gcc version 10.2.1 20201103 (GNU Toolchain for the A-profile
Architecture 10.2-2020.11 (arm-10.16))) #4 Fri Oct 18 17:07:30 CST 2024 arm, fwver: v1.10
// Configuration information. These values correspond to the configuration values of the
rockchip_suspend node in the kernel dts, where "cnt" indicates how many times it has entered
suspend.
gcfg: cnt=2 global=0x000002b8, pwren=0x0000c2c0, wakeup=0x20000008, dbg=0x00000180, idle: 0,
0, 0
// RC clock information, which can be ignored.
rc400: hz=362900, div=6170/65319
// The expanded information of the suspend configuration. Corresponding to the configuration
content of the rockchip_suspend node in the kernel dts, it is expanded here as a string for
easy reading.
suspend-mode: armoff_logoff, 24m_osc_dis, 32k_clk, 32k_src_rc, pwm0_ch0_regulator
debug-mode: timeout_3s
pwren-core: gpio0_a2 (act:low)
pwren-sleep: gpio0_a0 (act:low)
wakeup-en: gpio0_int, pmu_timeout_int
// Configuration of 'sleep-io-config':
//  - If the key-value pair is illegal, there will be a similar display: "ff950210
0c000800(invalid)".
//  - If the key-value pair exceeds 16 groups, there will be a similar display: "ff950210
0c000800(overflow)".
sleep-io:
  ff950208 0c000800
  ff950200 00c00080
// 0123... represents the software steps during the suspend process. After printing "wfi",
the entire SoC fully enters low-power suspend, waiting for an external interrupt to wake up
the system.
0123g4567|wfi
```

RK3506 System Wake-up Print (only arm power off):

```
// |654... represents the software steps after waking up. Note: If it is a Logic power-off
wake-up, there will also be a DDR bin print before this information.
|6543g210
// Wake-up source and interrupt number.
wakeup-pmu: gpio0_int
wakeup-gpio: gpio0_c1
wakeup-irq: 32
// The cumulative duration of the system in suspend (the duration of this round of suspend)
Wfi-total: 3.441s (this: 1.420s)
```

RK3506 System Wake-up Print (arm+logic power off):

```
// After waking up, there is a DDR print first
DDR c7d3dcd3f0 typ 24/10/11-16:20:18,fwver: v1.02
tREFI:4x, sr_idle:93, pd_idle:13
ES
get rd_skew=0x2a, wr_skew=0x2a
PHY drv:clk:60,ca:160,DQ:80,odt:480
vrefinner:50%, vrefout:50%
dram drv:40,odt:0
sr_dq:0, sr_ca:0, sr_clk:0
rg:0x19-0x0-0x2, 0xd-0x0-0x2,status:a007
DDR3, 750MHz
BW=16 Col=10 Bk=8 CS0 Row=13 CS=1 Size=128MB
jump 0x68f4
// Print of the wake-up process code
|6543g210
wakeup-pmu: pmu_timeout_int

Wfi-total: 1394.184s (this: 4.979s)
```

4. GPIO Troubleshooting

When debugging IO power consumption or other issues, users may need to inspect the state of each GPIO after the system has fully entered suspend. `PinTool` can convert printed GPIO register values into a visualized view for easier reading, as shown below:

Register dump (used as the tool input):

```
ioc0:
ff950000: 00007707 00000077 00007777 00000777
ff950010: 00000000 00001170
ff950100: 00000707 00000307 00000707 00000707
ff950110: 00000707 00000707 00000707 00000707
ff950120: 00000707 00000707 00000307 00000707
ff950200: 0000a088 00000002 000052a1
ff950300: 000000ff 000000ff 000000df
ff950400: 000000ff 000000ff 000000df
```

```
ff950500: 00000000 00000000 00000000
ff950600: 0000ffff 0000ffff 0000ffff
ff950700: 00000000 00000000 00000000
ff950800: 00000000
ff950830: 0000072c

gpio0:
ff940000: 00000000 00000000 00008082 00000000
ff940010: 0000ffff 000001ff 0000ffff 000001fd
ff940020: 00000040 00000000 00000000 00000000
ff940030: 00000000 00000002 00000000 00000000
ff940040: 00000000 00000000
ff940048: 0000013f
ff940060: 00000000 00000000
ff940070: 00c36035
ff940080: 00000000 00000000
ff940100: 0000ffff 000001ff 00000000
ff940110: 00000000 00000000 00000000 00000000
ff940120: 00000000 00000000
......
```

Converted output from the tool:

```
==========================GPIO0A0==========================
GPIO0A0_SEL                   |   7: RM_IO0
GPIO0A0_RMIO                  |  78: PMU_SLEEP
GPIO0A0_DDR                   |   0: input
GPIO0A0_DR                    |   0: low
GPIO0A0_DS                    |   7: level2
==========================GPIO0A1==========================
GPIO0A1_SEL                   |   0: GPIO0_A1
GPIO0A1_RMIO                  |   0: NO_FUNC
GPIO0A1_DDR                   |   1: output
GPIO0A1_DR                    |   0: low
GPIO0A1_DS                    |   7: level2
==========================GPIO0A2==========================
GPIO0A2_SEL                   |   7: RM_IO2
GPIO0A2_RMIO                  |  79: CORE_POWER_OFF
GPIO0A2_DDR                   |   0: input
GPIO0A2_DR                    |   0: low
GPIO0A2_DS                    |   7: level2
==========================GPIO0A3==========================
GPIO0A3_SEL                   |   7: RM_IO3
GPIO0A3_RMIO                  |  32: PWM0_CH2
GPIO0A3_DDR                   |   0: input
GPIO0A3_DR                    |   0: low
GPIO0A3_DS                    |   3: level1
==========================GPIO0A4==========================
GPIO0A4_SEL                   |   7: RM_IO4
GPIO0A4_RMIO                  |  19: I2C2_SCL
GPIO0A4_DDR                   |   0: input
GPIO0A4_DR                    |   0: low
GPIO0A4_DS                    |   7: level2
```

```
=========================GPIO0A5===========================
GPIO0A5_SEL                    |   7: RM_IO5
GPIO0A5_RMIO                   |  20: I2C2_SDA
GPIO0A5_DDR                    |   0: input
GPIO0A5_DR                     |   0: low
GPIO0A5_DS                     |   7: level2
=========================GPIO0A6===========================
GPIO0A6_SEL                    |   0: GPIO0_A6
GPIO0A6_RMIO                   |   0: NO_FUNC
GPIO0A6_DDR                    |   0: input
GPIO0A6_DR                     |   0: low
GPIO0A6_DS                     |   7: level2
=========================GPIO0A7===========================
GPIO0A7_SEL                    |   0: GPIO0_A7
GPIO0A7_RMIO                   |   0: NO_FUNC
GPIO0A7_DDR                    |   1: output
GPIO0A7_DR                     |   0: low
GPIO0A7_DS                     |   7: level2
=========================GPIO0B0===========================
GPIO0B0_SEL                    |   7: RM_IO8
GPIO0B0_RMIO                   |  59: SAI1_MCLK
GPIO0B0_DDR                    |   0: input
GPIO0B0_DR                     |   0: low
GPIO0B0_DS                     |   7: level2
......
```

How to use the tool:

1. Refer to the earlier "Debug Configuration" section and enable `RKPM_DBG_REG2`. After the system enters suspend, register values, including GPIO-related ones, will be printed. For example:

   ```
   0123g4567#58====== suspend-finish:
   cru:
   ff9a0280: 00000001
   ff9a0300: 000005c0 00000b67 00000229 00002a00
   ff9a0310: 00000055
   ff9a0314: 0008007d 00200177 00c00c35 00000000
   ff9a0324: 03723d09 00000000 00800c35 00070064
   ff9a0334: 01290c80
   ff9a033c: 000002a1 00000004
   ff9a0348: 00002185 000013a7
   ff9a0354: 00001000 00000001 00000000
   ff9a0374: 00000001 00000000 00000000 00000410
   .....

   ioc0:
   ff950000: 00007707 00000077 00007777 00000777
   ff950010: 00000000 00001170
   ff950100: 00000707 00000307 00000707 00000707
   ff950110: 00000707 00000707 00000707 00000707
   ff950120: 00000707 00000707 00000307 00000707
   ff950200: 0000a088 00000002 000052a1
   ff950300: 000000ff 000000ff 000000df
   ```

```
ff950400: 000000ff 000000ff 000000df
ff950500: 00000000 00000000 00000000
ff950600: 0000ffff 0000ffff 0000ffff
ff950700: 00000000 00000000 00000000
ff950800: 00000000
ff950830: 0000072c

gpio0:
ff940000: 00000000 00000000 00008082 00000000
ff940010: 0000ffff 000001ff 0000ffff 000001fd
ff940020: 00000040 00000000 00000000 00000000
ff940030: 00000000 00000002 00000000 00000000
ff940040: 00000000 00000000
ff940048: 0000013f
ff940060: 00000000 00000000
ff940070: 00c36035
ff940080: 00000000 00000000
ff940100: 0000ffff 000001ff 00000000
ff940110: 00000000 00000000 00000000 00000000
ff940120: 00000000 00000000
......

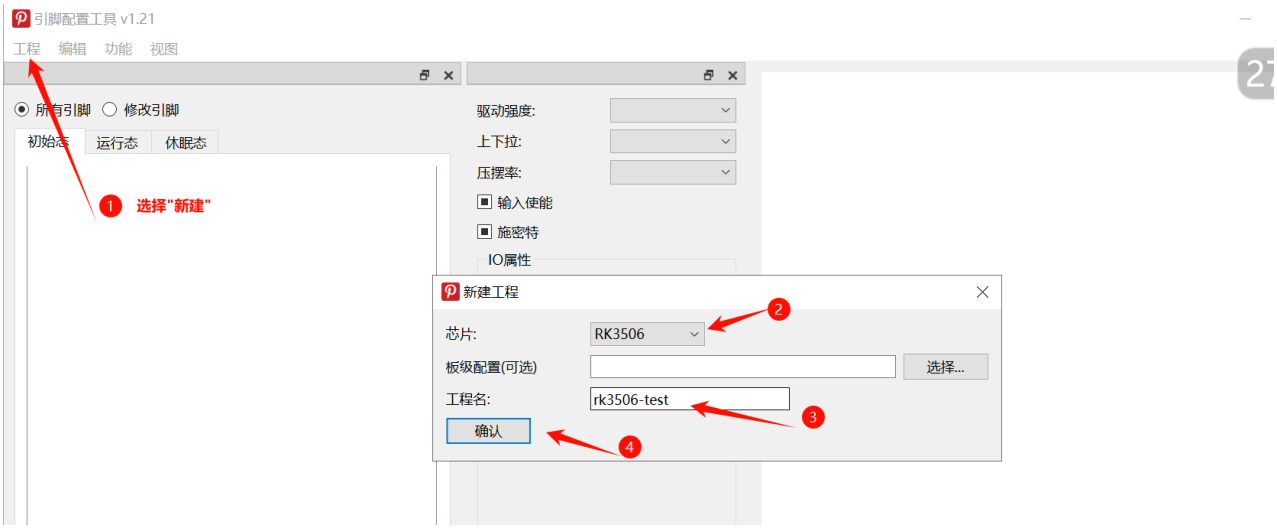
ff901110: 00000000
ff901120: 00000000
ff901128: 00000000
ff901130: 0000001f
ff901140: 00000000
|wfi
```

2. Copy all of the printed content above into a txt file, such as `gpio.txt`.
3. Open `PinTool` (recommended version >= v1.22).

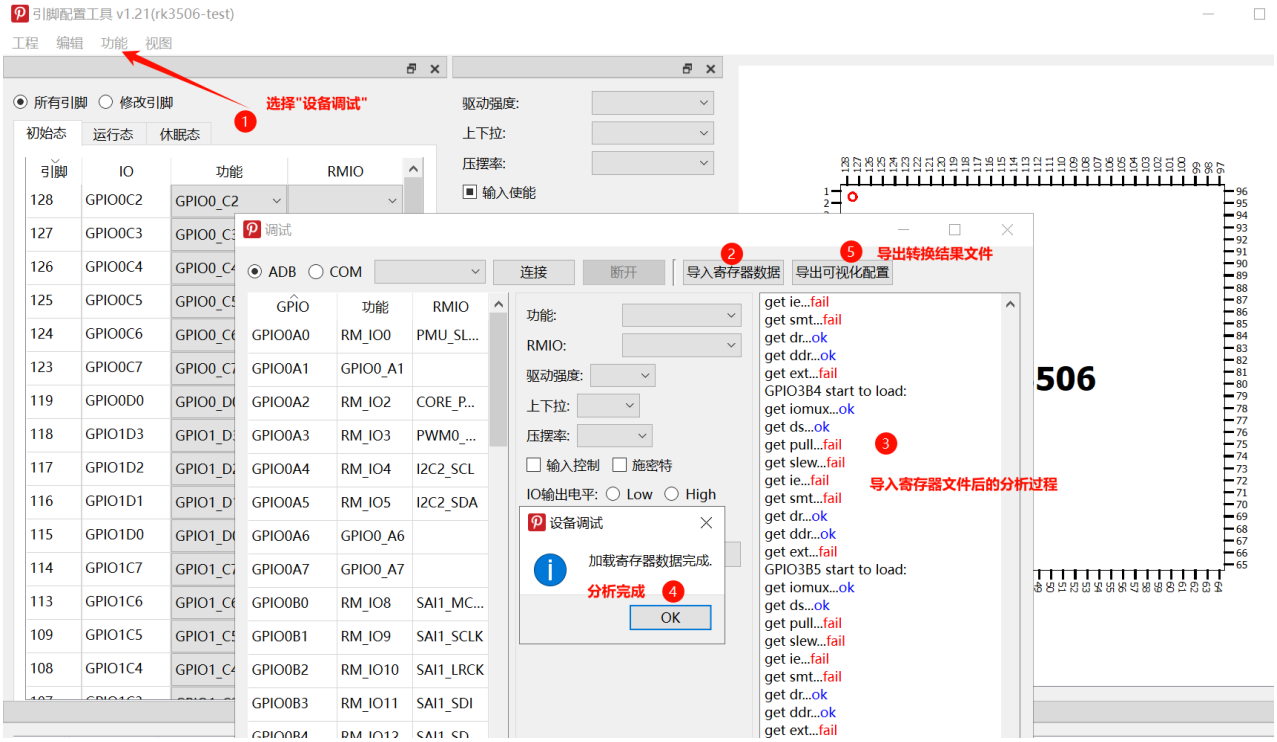
   Step 1: Create an RK3506 workspace.



名称	修改日期	类型	大小
chip	2025/2/20 14:34	文件夹	
lang	2024/12/16 11:10	文件夹	
log	2026/5/29 14:34	文件夹	
res	2024/12/16 11:10	文件夹	
config.json	2025/2/20 14:35	JSON 文件	1 KB
PinTool.exe	2025/2/20 15:15	应用程序	63,564 KB
PinTool_UserManual_v1.21.pdf	2025/2/20 14:52	PDF 文件	1,427 KB
revision.log	2025/2/20 14:34	LOG 文件	1 KB



Step 2: Import the register value file `gpio.txt` and get the converted result file.



5. Troubleshooting

Enable debug information:

1. Kernel defconfig:

```
CONFIG_PM=y
CONFIG_PM_DEBUG=y
CONFIG_PM_SLEEP_DEBUG=y
CONFIG_RK_CONSOLE_THREAD=n
```

2. Append to kernel DTS `bootargs`:

```
no_console_suspend initcall_debug debug pm_debug_messages
```

Troubleshooting approach:

- If the system hangs and cannot wake up after printing `0123g4567|wfi`, please check:
  - Are the power supply states of each rail as expected?
  - Use an oscilloscope to capture the level changes of the pwrctrl pin during the entire suspend and wake-up process, and check if they are as expected.
- If the ARM+LOGIC power-off mode is unstable, is the ARM power-off mode stable?

  > Note: Synchronously modify the regulator suspend state in the kernel dts.

- Is it stable when using a higher version of rk3506_tee_vx.yy.bin?
- Is it stable when enabling RKPM_DBG_CLK_UNGATE?
- Is it stable when enabling RKPM_DBG_PLLS_ALIVE?
- Is it stable when removing CPU1/2 from DTS, leaving only CPU0?